📖 Manejo de Estado con Provider (Nivel Intermedio)

1. **Definición**

- **Provider** es un paquete oficial recomendado por la comunidad Flutter para manejar estado de manera **simple, eficiente y escalable**.
- Se basa en **InheritedWidget**, pero lo simplifica y lo hace más práctico.
- Permite **compartir datos y lógica de negocio** entre múltiples widgets sin necesidad de pasar parámetros manualmente.

---

2. **Teoría**

- **Provider** actúa como un **contenedor de estado** que se coloca en la raíz de la aplicación.
- Los widgets hijos pueden **escuchar** ese estado y **reaccionar automáticamente** cuando cambia.
- Se usa junto con **ChangeNotifier**, una clase que notifica a los widgets cuando hay cambios.

Conceptos clave:

- **ChangeNotifier**: Clase que mantiene datos y notifica cambios.
- **ChangeNotifierProvider**: Expone un ChangeNotifier a todo el árbol de widgets.
- **Consumer / context.watch / context.read**: Formas de acceder al estado.

---

3. **Escenarios donde aplicar Provider**

- **Carrito de compras**: compartir productos seleccionados entre catálogo y carrito.
- **Sesión de usuario**: mantener datos de login accesibles en toda la app.
- **Configuraciones globales**: tema claro/oscuro, idioma, preferencias.
- **Datos compartidos**: listas, contadores, formularios que afectan varias pantallas.

---

4. **Cómo aplicarse**

1. Crear una clase que extienda **ChangeNotifier**.
2. Definir variables de estado y métodos para modificarlas.
3. Usar **notifyListeners()** para avisar que el estado cambió.
4. Envolver la app con **ChangeNotifierProvider**.
5. Acceder al estado desde widgets hijos con **Consumer** o `context.watch`.

💻 Ejemplo básico: Contador con Provider

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

// ---------------- Modelo de estado ----------------
class CounterModel extends ChangeNotifier {
  int _counter = 0;

  int get counter => _counter;

  void increment() {
    _counter++;
    notifyListeners(); // Notifica a la UI
  }

  void decrement() {
    _counter--;
    notifyListeners();
  }
}

// ---------------- App principal ----------------
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CounterModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Contador con Provider',
      theme: ThemeData(primarySwatch: Colors.green),
      home: const CounterScreen(),
    );
  }
}

// ---------------- Pantalla principal ----------------
class CounterScreen extends StatelessWidget {
  const CounterScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final counterModel = Provider.of<CounterModel>(context);

    return Scaffold(
```

```
      appBar: AppBar(title: const Text('Contador con Provider')),
      body: Center(
        child: Text(
          'Valor actual: ${counterModel.counter}',
          style: const TextStyle(fontSize: 24),
        ),
      ),
      floatingActionButton: Row(
        mainAxisAlignment: MainAxisAlignment.end,
        children: [
          FloatingActionButton(
            heroTag: 'decrement',
            onPressed: counterModel.decrement,
            child: const Icon(Icons.remove),
          ),
          const SizedBox(width: 20),
          FloatingActionButton(
            heroTag: 'increment',
            onPressed: counterModel.increment,
            child: const Icon(Icons.add),
          ),
        ],
      ),
    );
  }
}
```

---

5. **Explicación paso a paso del ejemplo**

1. **CounterModel**:
   - Contiene la variable `_counter`.
   - Métodos `increment` y `decrement` modifican el estado.
   - `notifyListeners()` avisa a la UI que debe actualizarse.
2. **ChangeNotifierProvider**:
   - Envuelve la app y expone `CounterModel` a todos los widgets hijos.
3. **Provider.of(context)**:
   - Permite acceder al estado desde la pantalla.
   - Cada vez que cambia `_counter`, la UI se reconstruye automáticamente.